基于 MongoDB 的问卷系统

第一阶段项目完成报告

GitHub: https://github.com/Robert4063/Contemporary-Data-Management-Systems_26_fall；分支：project1

在这次任务中，我们要完成一个可以创建、分享、填写和统计的在线问卷系统。创建者先设置题目、校验和跳转条件，再把链接发给其他登录用户；填写时，后端根据 MongoDB 中的配置判断答案是否合法，最后把实际答卷保存下来。与只完成几个页面相比，这次更需要说明数据为什么这样组织，以及规则发生变化后，原来的功能能不能继续使用。

1. 项目目标与需求分析

在进行设计之前，我们会遇到第一个问题：问卷系统的核心应该放在哪里？把题目直接写在页面里是一个很自然的想法，但是这样每增加一份问卷就要改代码，也无法让创建者自己设置规则。因此，这次把用户、问卷配置和答卷分别保存，前端负责交互，后端负责权限、校验、跳转和统计。

表1 第一阶段的实现范围

部分	需要完成的内容
账号与管理	注册、登录、自己的问卷列表；仅创建者可以编辑、发布、关闭和查看统计。
题目与填写	单选、多选、文本、数字四种题型；按配置校验、条件跳转和保存答卷。
发布与时间	创建时记录时间；截止必填；支持分享链接、发布确认、直接发布和截止后的只读查看。
提交与统计	匿名仍需登录；默认每账号一次，可预先允许多次；支持重试去重、整卷与单题统计。

技术上选择 Python、FastAPI、PyMongo、MongoDB 和原生 HTML/CSS/JavaScript。业务数据全部存入 MongoDB，没有使用关系数据库或现成问卷框架；本设备填写进度与登录凭证只是浏览器缓存，不作为已提交答卷参与统计。

2. MongoDB 数据结构设计

只要看了问卷本身的形式，我们就会很自然地问出第二个问题：哪些内容应该放在一起？题目、选项、校验和跳转通常一起读取，也在发布前一起确定，因此把它们嵌入 surveys。与这些配置相比，答卷会不断增加，如果也放进问卷数组，文档就会越来越大，所以 responses 独立保存。

表2 集合划分与主要字段

集合	主要字段	嵌入或引用的理由
users	username、password_hash、created_at	账号有独立生命周期，用户名唯一。
sessions	user_id、token_hash、expires_at	引用用户，支持过期和退出撤销。
surveys	owner_id、status、deadline、questions、revision	引用创建者；嵌入题目、选项、validation 和 rules。
responses	survey_id、answers、path、submitted_at	引用问卷；每次合法提交独立保存，避免大数组。

这里没有把所有内容机械拆成表式结构。不同题型的约束并不相同，文档可以保存完整配置；同时，独立答卷也避免问卷文档随填写人数持续增长。这种选择适合当前的读写方式，但不意味着关联可以不检查，owner_id 和 survey_id 仍由后端控制。

确定集合之后，我们还需要考虑查询和并发。users 的 username 建唯一索引，sessions 的 token_hash 唯一，并给 expires_at 建 TTL 索引；问卷按 owner_id 和 created_at 查询，答卷按 survey_id 和 submitted_at 查询。禁止重复时，survey_id 与 respondent_key 的部分唯一索引限制每账号一次；同次提交则由 submission_key 的部分唯一索引保护。

MongoDB 的类型校验器检查核心字段类型，复杂的题目范围和跳转目标由 Pydantic 与业务解释器检查。TTL 只负责清理过期会话，认证时仍然比较 expires_at。schema_version 表示数据格式，revision 用于并发控制；它们都不等于完整的历史版本。

3. 系统实现与问卷生命周期

准备好数据结构之后，我们来实现请求流程。前端向 FastAPI 发送 JSON，models.py 先检查请求结构，security.py 验证登录，main.py 检查权限与状态，再由 logic.py 或 statistics.py 完成业务计算，最后通过 PyMongo 访问数据库。前后端由同一个服务提供，浏览器不直接连接 MongoDB。

3.1 登录和权限

注册时用户名统一为小写，密码以随机盐 PBKDF2-HMAC-SHA256 保存。登录后生成随机 Bearer Token，sessions 只保存 Token 的哈希和过期时间。每次受保护请求都重新验证会话；管理问卷时，同时用问卷 ID 和当前 owner_id 查询，因此不能靠修改 URL 管理别人的问卷。

匿名填写应该怎样理解呢？这次仍要求先登录，匿名只控制答卷是否直接保存 respondent_id。禁止重复时，后端依然保存每问卷、每账号的 HMAC 去重键，所以切换匿名不能绕过次数限制。这个做法便于控制次数，但服务器仍能验证关联，不应称为绝对不可追踪的匿名。

3.2 创建、保存和发布

点击创建问卷后，服务器立即建立空草稿并记录不可由客户端修改的 created_at。初建空草稿可以暂缺截止，正式保存和发布则必须提供含时区的合法时间，不得早于创建时间，而且必须晚于服务器当前时间。这样就不会等到最后发布，才把发布时间误当成创建时间。

表3 问卷状态与操作边界

状态	登录后查看	填写与提交	允许的修改
草稿 draft	仅创建者管理	不公开接收	编辑题目、规则与策略。
发布且未截止	持链接可查看	允许，仍校验次数	标题说明、合法截止。
自然截止	全部只读题目	拒绝新增答卷	未手动关闭时可延长截止。
手动关闭 closed	全部只读题目	拒绝新增答卷	不能改题或延长截止。

保存时携带 revision，只有草稿状态和版本同时匹配才更新，避免旧页面覆盖新内容。普通发布重新校验已存配置；直接发布则在一次文档原子更新中保存当前配置并发布。两个前端入口都先确认，取消不发发布请求，但确认框不能代替后端的权限检查。

自然截止由 deadline 动态判断，不另写 expired 状态。status 接口在登录后返回只读题目和 can_fill，而填写配置、下一题和新提交仍检查可填写状态。因此，截止后能够进入页面并返回工作台，不代表还可以继续提交。

4. 校验与跳转设计

有了题目之后，我们还会遇到一个问题：页面上已经限制了输入，后端是不是可以直接相信答案？实际上，用户可以直接调用接口，也可能在填写途中修改答案，所以页面检查只能提供提示，最终判断必须来自 MongoDB 中保存的配置。

表4 题目配置与服务端校验

题型	保存的规则	后端实际检查
单选 single	required、options	答案必须是本题的有效选项 ID。
多选 multiple	required、min_choices、max_choices	必须是有效 ID 数组；不重复，数量符合上下界。
文本 text	required、min_length、max_length	必须是字符串；检查空白与文本长度。
数字 number	minimum、maximum、integer_only	排除布尔、字符串和非有限值；检查范围、整数及绝对值上限。

配置本身也需要检查。最少选择数不能大于最多，最多不能超过选项数；文本和数字上下界不能倒置，题目 ID 与选项 ID 不能重复。发布前还要检查至少有一道题、截止合法、目标存在。非法配置或答案返回 422，并说明对应字段或题目，而不是保存后再让统计处理错误数据。

4.1 用配置表示跳转

如果把“第1题选否就跳第3题”写在程序分支里，其他问卷就无法复用。为了处理这个问题，我在需求中要求跳转可配置，AI 将每条规则表示为 op、value 和 target，并用 jump_enabled 控制是否执行。例如，单选题可以保存下面的配置：

```
jump_enabled: true
rules: [{op: "eq", value: "no", target: "q3"}]
```

它只说明答案等于选项 ID no 时进入 q3，并不依赖题干写了什么。选项显示 A、B、C，内部仍使用稳定 ID；删除或上移后重新编号，不改变原有引用。目标题目改名时更新下拉文字，保存的 target 不变。

表5 跳转操作及数量限制

类型	条件	当前配置限制
单选	选项匹配；兼容旧 eq/ne	按选项设置目标，同一选项不重复配置 eq。
多选	contains_any / all / none	每题最多一条；“全都不包含”表示与配置项无交集。
数字	eq/ne、gt/gte、lt/lte、between	每题最多一条；between 包含两端。
文本	无条件跳转	不允许启用。

4.2 从首题重放实际路径

那么，怎样判断用户有没有跳过必答题呢？walk 每次从第一题开始，先校验答案，再执行第一条命中规则，没有命中就顺序进入下一题。目标只能是后续题或 END，避免循环；前端提交的路径不作为依据。比如 q1 选 no 跳 q3，q2 没有经过，就不要求回答；如果请求仍夹带 q2，后端反而拒绝。

5. 测试结果与分析

完成实现之后，我们来检查这些规则是否真的生效。测试使用真实 MongoDB 的随机独立测试库，浏览器部分由 Playwright 驱动 Chrome；测试结束只清理本次测试库。精简目录调整后重新运行完整回归，结果如下：

表6 当前完整回归结果

测试文件	范围	用例数	通过
test_system.py	账号、创建、增题、权限、提交、并发与统计	44	44
test_improvements.py	截止、跳转开关、幂等、只读与直接发布	46	46
test_logic.py	操作符、边界、空值与兼容	20	20
test_browser.py	真实页面、共享登录、交互与竞态	16	16
合计	失败 0，跳过 0	126	126

本次执行完成时间为 2026-10-22T05:32:34 UTC，总耗时约 170.26 秒；环境为 Python 3.13.2、MongoDB 8.3.11 和 Chrome 155.0.8059.80。保留 1 条 Starlette/AnyIO 依赖弃用警告，进程退出码为 0。逐例输入、预期、实际输出与结果见“03_测试用例/测试用例与结果.md”，不把不同轮次的通过数相加。

表7 关键测试的输入与实际结果

检查	输入或步骤	预期与实际结果
创建与增题	创建草稿，添加四类题，再编辑和发布	创建、增题均为 201；发布成功，旧版本更新被拒绝。
路径与校验	q1=a 跳 q3；q3 包含 a/b 跳 q5；q5≥18 结束	实际 path 为 q1、q3、q5；夹带跳过题答案返回 422。
多选新条件	配置 contains_none；答案与规则选项无交集	命中唯一规则；保存第二条多选规则被拒绝。
账号并发	同账号8路；再由8个不同账号提交	同账号1个201、7个409；其他账号8个201，共9份。
统计样本	单选 a/a/b，多选 [a,b]/[a]，数字 10/20	整卷3份；单选 a=2/b=1，多选 a=2/b=1；数字均值15。
截止只读	登录后读取截止或关闭问卷，再尝试新提交	详情200，显示全部只读题目；新填写与提交拒绝。

可以看到，单选统计的是每个选项对应的答卷数，多选先展开数组再累计次数。统计样本有3份答卷，但数字只有2次回答，因此平均值是15，而不是把未回答当成0；完全没有数字答案时返回 null。允许多次填写时，这里得到的是答卷人次，不是自然人的去重人数。

整体来看，测试覆盖了初始需求规定的创建、添加题目、跳转、校验、提交和统计，也覆盖后续修正。但测试通过并不代表所有环境都已经验证。截止场景使用隔离库和时钟模拟，不能说成真实等待；Chrome 和390像素视口已经检查，其他浏览器、真实手机、压力与容灾没有实测。

6. AI 使用说明

这次任务的 AI 使用分为两个阶段：先与 ChatGPT 讨论任务和数据库思路，再用 Codex 完成工程实现与修改。具体模型版本没有记录，因此这里只按实际工具区分。下面按照作业要求，分别说明 AI 帮了什么、AI 做错了什么、自己改了什么和自己设计了什么。

6.1 AI 帮了什么

最开始，我认为重点只是设计数据库，后端主要校验和提示错误。GPT补充了权限、状态、跳转、保存和统计等职责，帮助我区分业务实体与集合，以及题目配置与实际答案，并解释按 type 选择通用校验器的思路。这个阶段得到的是设计说明、结构示意和教学代码片段，不是已经验收的完整系统。

表8 AI工具、实际提示与得到的内容

工具与阶段	实际提示或输入	得到的内容
ChatGPT：前置学习	“给我一个后端代码示例。”	问卷JSON、MongoDB连接与SubmitRequest等片段；粘贴记录中的Python代码不完整，没有实测运行依据。
Codex：工程实现	原始作业与完整实现要求；后续追加截止、跳转、登录和发布等反馈。	生成main.py、models.py、logic.py、db.py、前端和测试；通过终端、MongoDB、Pytest与Chrome验证并修订。

Codex把这些想法落实为四集合、配置模型、路径解释器、认证、接口和页面，并编写自动化测试与项目文档。实际代码和文件写入由代理执行；GPT给出的合法/非法输入只有预期，不能混入后续126项真实回归的结果。

6.2 AI 做错了什么

AI的首轮实现并不完整。后续发现严格类型边界不足、内部选项ID直接显示、截止后卡在登录页、新标签重复登录，以及旧会话迟到401清除新登录等问题；报告还曾混入多个版本的测试数字，提交材料也过于冗杂。针对这些问题，依据反馈继续修改实现并补回归，旧响应竞态确实先复现失败再修复。环境中的WinError 5另行排查，不能把环境失败算成业务通过。

6.3 自己改了什么

我主要修改的是需求、交互规则和交付要求：补充创建时间与截止必填，要求单选逐选项跳转、多选“全都不包含”、启用开关、字母编号、目标标题联动、截止只读与返回工作台、直接发布确认和已有登录复用。材料方面，我要求按四类精简、文档与报告分开，并调整PDF文风与密度。对应Python和JavaScript由Codex实际修改；我提出问题、选择方案和检查反馈，不把代理编码写成自己手工改代码。

6.4 自己设计了什么

前置讨论中，我提出用户、问卷、题目、答卷和统计结果五类实体，并明确选择“规则存入问卷、前端辅助提示且后端最终校验”和“题目嵌入问卷”。GPT帮助纠正了实体不等于集合、surveys不保存实际答卷等认识。后续我确认登录且持链接可填、默认每账号一次、发布后题目规则冻结、允许修正标题说明，以及本设备暂存和同次重试只存一份。我的设计贡献主要是实体识别、存储方向及行为约束；具体Schema字段、索引和解释器由AI协助细化并实现。

完整过程另见同目录AI使用日志，逐条保留工具、Prompt、Result、是否修改和修改内容。前置GPT记录依据本人粘贴的对话，不能把未取得的完整示例代码补写成历史事实。

7. 遇到的问题与解决方法

那么，这些修改到底解决了什么问题呢？与最初相比，主要变化不是增加了多少按钮，而是把前端展示、后端规则和数据库约束对齐。几个比较典型的问题如下：

表9 问题、原因与修正

问题	原因	解决办法
截止后停留在登录页	把查看详情与允许填写混成同一条件。	状态详情在登录后只读返回；填写许可单独校验，并提供返回工作台。
删除选项后编号混乱	内部 ID 与显示名称没有分开处理。	保留稳定 ID，各页面按当前数组顺序显示 A/B/C；同步目标标题。
网络重试可能重复保存	只做页面防连点不能覆盖丢包和并发。	同次 UUID、内容哈希和唯一索引共同判断；默认账号次数另行约束。
新登录被旧401清除	迟到响应仍按当前会话处理。	响应关联发起时的 Token，忽略旧会话结果；增加真实浏览器回归。
提交附件过于冗杂	完整开发归档与分类副本同时放入。	只保留代码、四份设计说明、测试和报告；AI过程写入报告，原始提示另留日志。

继续分析会发现，前端和后端各做一次检查并不是重复劳动。前端减少无效操作，后端保证直接调用接口时仍受约束；唯一索引则处理两个请求同时到达的情况。三者对应的问题不同，所以不能用页面上“已经禁用”来代替真正的数据保护。

最终提交会重新执行完整校验。submission_id 标识同一次填写，作用域 HMAC 与内容哈希一起检查重试：同次同内容返回原结果，不再保存；同次换内容返回 409。新提交还会原子核对发布状态、版本和截止，再插入答卷。准入和插入没有跨集合事务，已准入的在途请求可能在关闭后完成，统计因此只认 responses，不把 admissions 当作答卷数。

8. 实现思考与后续扩展

在完成第一阶段之后会发现，更多功能并不必然带来更好的设计。比如发布后允许随意改题，看起来更方便，却会让已有答案与统计失去明确对应关系；这次因此冻结题目和策略，只开放标题说明与合法截止。第二阶段如果真的需要改题，应在现有模块上增加版本或迁移方案，而不是直接覆盖旧配置。

当前仍有明确边界：未部署公网与 HTTPS，未验证生产 MongoDB 认证、限流、备份容灾或压力表现；统计尚无海量文本分页和跨查询快照，账号一次也不能阻止同一人注册多个账号。本设备暂存不能跨设备恢复，禁用存储回退只做过代码检查。这些没有因为126项通过而自动完成，应根据真实新需求继续处理。

最后，交付按原作业保留四类：01_代码包含前后端、依赖、环境示例、Schema和启动说明；02_文档包含系统、数据库、API和关键逻辑；03_测试用例包含可复现测试及输入、预期、实际结果；04_项目完成报告包含本报告与AI日志。已有 MongoDB 服务可以复用，启动前安装依赖并配置数据库地址、库名和稳定密钥。

记录说明：表6保留精简目录回归的原始时间与结果。本轮README编写另完成全新环境安装、HTTP核验及完整回归，结果见根目录README和测试材料，各轮不累加；原始提示及修改过程见AI使用日志。